East West University

Department of Computer Science and Engineering (CSE)
Semester: (Spring, Year: 2026), B.Sc. in CSE

Code Smell Lab

Group: 2

Course Code: *CSE423*

Course Title: *Software Architecture*
Section: 01

Submitted to:

Ahmed Adnan
Lecturer, Department of Computer Science and Engineering

Students Details

Name	ID
Azahar Mahmud Chowdhury Rafi	2021-3-60-019
Sheikh Md Rakibul Hasan	2022-3-60-009
Ahnaf Anan	2023-1-60-089

Submission Date: 14th March, 2026

Code Smells Addressed:

- **Inappropriate Naming:** Renamed unclear variable names (e, en, s, c, ss, cs) to meaningful names (enrollment, student, course, studentService, courseService).
- **Comments Smell:** Removed comments acting as deodorant by extracting methods with self-documenting names in BackupService and CourseManagementHelper.
- **Long Method:** Broke down the lengthy getStudentTranscript() method in StudentService into seven focused, single-responsibility methods.
- **Primitive Obsession:** Replaced magic numbers with named constants (valid_grade_threshold) and extracted inline lambda expressions into descriptive methods (getNoFilterPredicate, hasValidGrade) to improve code clarity and maintainability.

Refactoring Principles Applied:

- **Extract Method:** Applied to break down long methods (getStudentTranscript), eliminate comments (BackupService), and replace primitive lambda expressions with named methods (CourseManagementHelper, ReportService).
- **Rename Method/Variable:** Systematically renamed unclear identifiers to improve code readability.
- **Introduce Constants:** Replaced magic numbers with named constants (valid_grade_threshold = 0.0) to eliminate primitive obsession and improve code maintainability.

Git Repository Link [Initial Main Code]: https://github.com/Azahar-Mahmud/student-enrollment-system-refactored/commit/661e1dda4498167ad6d4f02c6208308e97385bb9

Code Smell: Long Method

Files Affected: StudentService.java

Issue: The getStudentTranscript() method was building student profile, creating headers, formatting data, and calculating GPA.

1. **File:** src/edu/ccrm/service/StudentService.java

Before:

```
public String getStudentTranscript(String regNo) {
        Student student = findStudentByRegNo(regNo);
        StringBuilder transcript = new StringBuilder();

        transcript.append(student.getProfile()).append("\n\n");
```

```
        transcript.append("--- Academic Transcript ---\n");

        if (student.getEnrolledCourses().isEmpty()) {
            transcript.append("No courses enrolled yet.\n");
        } else {
            transcript.append(String.format("%-10s | %-30s | %-10s | %s\n",
"Code", "Course Title", "Credits", "Grade"));
            transcript.append("-----------------------------------------------
------------------\n");
            for (Enrollment en : student.getEnrolledCourses()) {
                Course course = en.getCourse();
                transcript.append(String.format("%-10s | %-30s | %-10d |
%s\n",
                        course.getCourseCode(), course.getTitle(),
                        course.getCredits(), en.getGrade()));
            }
        }
        transcript.append("-------------------------------------------------
---------------\n");
        transcript.append(String.format("GPA: %.2f\n",
calculateGpa(student)));

        return transcript.toString();
    }
```

After:
File: src/edu/ccrm/service/StudentService.java

```
package edu.ccrm.service;

import java.util.Comparator;
import java.util.List;
import java.util.stream.Collectors;

import edu.ccrm.domain.Student;
import edu.ccrm.exception.StudentNotFoundException;
import edu.ccrm.util.TranscriptFormatter;

public class StudentService {
    private final DataStore dataStore;
    private final TranscriptFormatter transcriptFormatter;

    public StudentService(DataStore dataStore) {
        this.dataStore = dataStore;
        this.transcriptFormatter = new TranscriptFormatter();
    }

    public void addStudent(Student student) {
        dataStore.addStudent(student);
    }

    public Student findStudentByRegNo(String regNo) {
        Student student = dataStore.getStudent(regNo);
```

```java
        if (student == null) {
            throw new StudentNotFoundException("Student with Registration No
'" + regNo + "' not found.");
        }
        return student;
    }

    public List<Student> getAllStudents() {
        return dataStore.getAllStudents().stream()
            .sorted(Comparator.comparing(Student::getRegNo))
            .collect(Collectors.toList());
    }

    public void updateStudentStatus(String regNo, Student.StudentStatus
status) {
        Student student = findStudentByRegNo(regNo);
        student.setStatus(status);
    }

    public String getStudentTranscript(String regNo) {
        Student student = findStudentByRegNo(regNo);
        return transcriptFormatter.format(student);
    }
}
```

File: src/edu/ccrm/util/ TranscriptFormatter.java

```java
package edu.ccrm.util;

import edu.ccrm.domain.Course;
import edu.ccrm.domain.Enrollment;
import edu.ccrm.domain.Student;

public class TranscriptFormatter {

    public String format(Student student) {
        StringBuilder transcript = new StringBuilder();
        appendStudentProfile(transcript, student);
        appendTranscriptHeader(transcript);
        appendCourseDetails(transcript, student);
        appendGpaSummary(transcript, student);
        return transcript.toString();
    }

    private void appendStudentProfile(StringBuilder transcript, Student
student) {
        transcript.append(student.getProfile()).append("\n\n");
    }

    private void appendTranscriptHeader(StringBuilder transcript) {
        transcript.append("--- Academic Transcript ---\n");
    }
```

```java
    private void appendCourseDetails(StringBuilder transcript, Student
student) {
        if (student.getEnrolledCourses().isEmpty()) {
            transcript.append("No courses enrolled yet.\n");
        } else {
            appendCourseTableHeader(transcript);
            appendCourseSeparator(transcript);
            appendCourseRows(transcript, student);
        }
    }

    private void appendCourseTableHeader(StringBuilder transcript) {
        transcript.append(String.format("%-10s | %-30s | %-10s | %s\n",
                "Code", "Course Title", "Credits", "Grade"));
    }

    private void appendCourseSeparator(StringBuilder transcript) {
        transcript.append("-------------------------------------------------
------------\n");
    }

    private void appendCourseRows(StringBuilder transcript, Student student) {
        for (Enrollment enrollment : student.getEnrolledCourses()) {
            Course course = enrollment.getCourse();
            transcript.append(String.format("%-10s | %-30s | %-10d | %s\n",
                    course.getCourseCode(), course.getTitle(),
                    course.getCredits(), enrollment.getGrade()));
        }
    }

    private void appendGpaSummary(StringBuilder transcript, Student student) {
        transcript.append("-------------------------------------------------
------------\n");
        transcript.append(String.format("GPA: %.2f\n",
calculateGpa(student)));
    }

    private double calculateGpa(Student student) {
        double totalPoints = 0;
        int totalCredits = 0;
        for (Enrollment enrollment : student.getEnrolledCourses()) {
            if (enrollment.getGrade().getGradePoint() >= 0) {
                totalPoints += enrollment.getGrade().getGradePoint()
                               * enrollment.getCourse().getCredits();
                totalCredits += enrollment.getCourse().getCredits();
            }
        }
        return totalCredits == 0 ? 0.0 : totalPoints / totalCredits;
    }
}
```

Code Smell: Primitive Obsession

File: src/edu/ccrm/service/ReportService.java

Issue: Using primitive values (magic numbers) instead of named constants reduces code readability and maintainability.

1.

Before:

```
private double calculateGpa(Student student) {
        double totalPoints = 0;
        int totalCredits = 0;

        for (Enrollment enrollment : student.getEnrolledCourses()) {
            if (hasValidGrade(enrollment)) {
                totalPoints += enrollment.getGrade().getGradePoint() *
enrollment.getCourse().getCredits();
                totalCredits += enrollment.getCourse().getCredits();
            }
        }
        return totalCredits == 0 ? 0.0 : totalPoints / totalCredits;
    }
```

After:

```
private static final double valid_grade_threshold = 0.0;
    private double calculateGpa(Student student) {
        double totalPoints = 0;
        int totalCredits = 0;

        for (Enrollment enrollment : student.getEnrolledCourses()) {
            if (hasValidGrade(enrollment)) {
                totalPoints += enrollment.getGrade().getGradePoint() *
enrollment.getCourse().getCredits();
                totalCredits += enrollment.getCourse().getCredits();
            }
        }
        return totalCredits == 0 ? 0.0 : totalPoints / totalCredits;
    }
    private boolean hasValidGrade(Enrollment enrollment) {
       return enrollment.getGrade().getGradePoint() >= valid_grade_threshold;
    }
```

Code Smell: Inappropriate Naming

Files Affected: EnrollmentService.java, StudentService.java, ReportService.java, CourseService.java, CourseCsvService.java, StudentCsvService.java, EnrollmentCsvService.java
Issue: Names given to variables and methods should be clear and meaningful. Single-letter or abbreviated names (e, en, c, ss, cs) make code harder to understand and violate the principle of intention-revealing code.

1.
Before:
File: : src/edu/ccrm/service/EnrollmentService.java:

```
public class EnrollmentService {
    private final StudentService studentService;
    private final CourseService courseService;

    public EnrollmentService(StudentService ss, CourseService cs) {
        this.studentService = ss;
        this.courseService = cs;
    }
```

After:

```
public class EnrollmentService {
    private final StudentService studentService;
    private final CourseService courseService;

    public EnrollmentService(StudentService studentService, CourseService
courseService) {
        this.studentService = studentService;
        this.courseService = courseService;
    }
```

2.

Before:

```
private void validateNoDuplicateEnrollment(Student student, String courseCode)
        throws DuplicateEnrollmentException {
        boolean alreadyEnrolled = student.getEnrolledCourses().stream()
            .anyMatch(e ->
e.getCourse().getCourseCode().getFullCode().equals(courseCode));

        if (alreadyEnrolled) {
            throw new DuplicateEnrollmentException(
                "Student " + student.getRegNo() + " is already enrolled in course
" + courseCode);
        }
    }

    private void validateCreditLimit(Student student, Course course)
        throws MaxCreditLimitExceededException {
        int currentCredits = student.getEnrolledCourses().stream()
            .filter(e -> e.getCourse().getSemester() == course.getSemester())
            .mapToInt(e -> e.getCourse().getCredits())
```

```
            .sum();

        int maxCredits = AppConfig.getInstance().getMaxCreditsPerSemester();
        if (currentCredits + course.getCredits() > maxCredits) {
            throw new MaxCreditLimitExceededException(
                "Cannot enroll. Exceeds max credit limit of " + maxCredits + "
for the semester.");
        }
    }

    private Enrollment findEnrollment(Student student, String courseCode) {
        return student.getEnrolledCourses().stream()
            .filter(en ->
en.getCourse().getCourseCode().getFullCode().equals(courseCode))
            .findFirst()
            .orElse(null);
    }
```

After:

```
private void validateNoDuplicateEnrollment(Student student, String courseCode)
        throws DuplicateEnrollmentException {
    boolean alreadyEnrolled = student.getEnrolledCourses().stream()
            .anyMatch(enrollment ->
                enrollment.getCourse().getCourseCode().getFullCode()
                    .equals(courseCode));
}

private void validateCreditLimit(Student student, Course course)
        throws MaxCreditLimitExceededException {
    int currentCredits = student.getEnrolledCourses().stream()
            .filter(enrollment -> enrollment.getCourse().getSemester()
                == course.getSemester())
            .mapToInt(enrollment -> enrollment.getCourse().getCredits())
            .sum();
}

private Enrollment findEnrollment(Student student, String courseCode) {
    return student.getEnrolledCourses().stream()
            .filter(enrollment ->
                enrollment.getCourse().getCourseCode().getFullCode()
                    .equals(courseCode))
            .findFirst().orElse(null);
}
```

3.

File: *File:* src/edu/ccrm/service/StudentService.java
Before:

```
for (Enrollment en : student.getEnrolledCourses()) {
                Course course = en.getCourse();
                transcript.append(String.format("%-10s | %-30s | %-10d | %s\n",
                        course.getCourseCode(), course.getTitle(),
                        course.getCredits(), en.getGrade()));
            }
```

And

```
for (Enrollment en : student.getEnrolledCourses()) {
            if (en.getGrade().getGradePoint() >= 0) {
                totalPoints += en.getGrade().getGradePoint() *
en.getCourse().getCredits();
                totalCredits += en.getCourse().getCredits();
            }
        }
```

After:

```
for (Enrollment enrollment : student.getEnrolledCourses()) {
                Course course = enrollment.getCourse();
                transcript.append(String.format("%-10s | %-30s | %-10d | %s\n",
                        course.getCourseCode(), course.getTitle(),
course.getCredits(), enrollment.getGrade()));
            }
```

And:

```
for (Enrollment enrollment : student.getEnrolledCourses()) {
            if (enrollment.getGrade().getGradePoint() >= 0) {
                totalPoints += enrollment.getGrade().getGradePoint() *
enrollment.getCourse().getCredits();
                totalCredits += enrollment.getCourse().getCredits();
            }
        }
```

4.

File: src/edu/ccrm/service/ReportService.java
Before:

```
private double calculateGpa(Student student) {
        double totalPoints = 0;
        int totalCredits = 0;
        for (Enrollment en : student.getEnrolledCourses()) {
            if (en.getGrade().getGradePoint() >= 0) {
                totalPoints += en.getGrade().getGradePoint() *
en.getCourse().getCredits();
                totalCredits += en.getCourse().getCredits();
            }
        }
        return totalCredits == 0 ? 0.0 : totalPoints / totalCredits;
    }
```

After:

```
private double calculateGpa(Student student) {
        double totalPoints = 0;
        int totalCredits = 0;
        for (Enrollment enrollment : student.getEnrolledCourses()) {
            if (enrollment.getGrade().getGradePoint() >= 0) {
                totalPoints += enrollment.getGrade().getGradePoint()
                             * enrollment.getCourse().getCredits();
                totalCredits += enrollment.getCourse().getCredits();
            }
        }
        return totalCredits == 0 ? 0.0 : totalPoints / totalCredits;
    }
```

5.

File: src/edu/ccrm/service/CourseService.java
Before:

```
public List<Course> getAllCourses() {
        return dataStore.getAllCourses().stream()
            .sorted(Comparator.comparing(c -> c.getCourseCode().getFullCode()))
            .collect(Collectors.toList());
    }
```

After:

```
public List<Course> getAllCourses() {
        return dataStore.getAllCourses().stream()
                .sorted(Comparator.comparing(
                    course -> course.getCourseCode().getFullCode()))
                .collect(Collectors.toList());
    }
```

6.

File: src/edu/ccrm/io/CourseCsvService.java

Before:

```
public void exportCourses() throws IOException {
        if (!Files.exists(dataDir)) {
            Files.createDirectories(dataDir);
        }

        Path coursesFile = dataDir.resolve("courses_export.csv");

        List<String> lines = dataStore.getAllCourses().stream()
            .map(c -> String.format("%s,%s,%d,%s,%d,%s",
                c.getCourseCode().getFullCode(), c.getTitle(),
                c.getCredits(), c.getSemester(),
                c.getInstructor().getId(), c.getDepartment()))
            .collect(Collectors.toList());

        lines.add(0, "courseCode,title,credits,semester,instructorId,department");
        Files.write(coursesFile, lines);
    }
```

After:

```
public void exportCourses() throws IOException {
        List<String> lines = dataStore.getAllCourses().stream()
                .map(course -> String.format("%s,%s,%d,%s,%d,%s",
                        course.getCourseCode().getFullCode(),
course.getTitle(),
                        course.getCredits(), course.getSemester(),
                        course.getInstructor().getId(),
course.getDepartment()))
                .collect(Collectors.toList());
    }
```

7.

File: src/edu/ccrm/io/StudentCsvService.java

Before:

```
public void exportStudents() throws IOException {
        if (!Files.exists(dataDir)) {
            Files.createDirectories(dataDir);
        }

        Path studentsFile = dataDir.resolve("students_export.csv");

        List<String> lines = dataStore.getAllStudents().stream()
            .map(s -> String.format("%d,%s,%s,%s,%s,%s",
                s.getId(), s.getRegNo(), s.getFullName(),
                s.getEmail(), s.getDateOfBirth(), s.getStatus()))
            .collect(Collectors.toList());
```

```
        lines.add(0, "id,regNo,fullName,email,dob,status");
        Files.write(studentsFile, lines);
    }
```

After:

```
public void exportStudents() throws IOException {
        if (!Files.exists(dataDir)) {
            Files.createDirectories(dataDir);
        }

        Path studentsFile = dataDir.resolve("students_export.csv");

        List<String> lines = dataStore.getAllStudents().stream()
            .map(student -> String.format("%d,%s,%s,%s,%s,%s",
                    student.getId(), student.getRegNo(),
student.getFullName(),
                    student.getEmail(), student.getDateOfBirth(),
student.getStatus()))
            .collect(Collectors.toList());

        lines.add(0, "id,regNo,fullName,email,dob,status");
        Files.write(studentsFile, lines);
    }
```

8.

File: src/edu/ccrm/io/EnrollmentCsvService.java

Before:

```
public void exportEnrollments() throws IOException {
        if (!Files.exists(dataDir)) {
            Files.createDirectories(dataDir);
        }

        Path enrollmentsFile = dataDir.resolve("enrollments_export.csv");

        List<String> lines = dataStore.getAllStudents().stream()
            .flatMap(s -> s.getEnrolledCourses().stream()
                .map(en -> String.format("%s,%s,%s",
                    s.getRegNo(),
                    en.getCourse().getCourseCode().getFullCode(),
                    en.getGrade())))
            .collect(Collectors.toList());

        lines.add(0, "regNo,courseCode,grade");
        Files.write(enrollmentsFile, lines);
    }
```

After:

```
public void exportEnrollments() throws IOException {
        if (!Files.exists(dataDir)) {
            Files.createDirectories(dataDir);
        }

        Path enrollmentsFile = dataDir.resolve("enrollments_export.csv");

        List<String> lines = dataStore.getAllStudents().stream()
            .flatMap(student -> student.getEnrolledCourses().stream()
                .map(en -> String.format("%s,%s,%s",
                           student.getRegNo(),
                       en.getCourse().getCourseCode().getFullCode(),
                       en.getGrade())))
            .collect(Collectors.toList());

        lines.add(0, "regNo,courseCode,grade");
        Files.write(enrollmentsFile, lines);
    }
```

Code Smell: Comments

Files Affected: BackupService.java, CourseManagementHelper.java

Issue: Comments were used as 'deodorant' instead of making code self-documenting.

1.

File: src/edu/ccrm/io/BackupService.java

Before:

```
public void createBackup() {
        AppConfig config = AppConfig.getInstance();
        Path sourceDir = Paths.get(config.getDataDirectory());

        String timestamp =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd_HH-mm-
ss"));
        Path backupParentDir = Paths.get(config.getBackupDirectory());
        Path backupDestDir = backupParentDir.resolve("backup_" + timestamp);

        try {
            if (!Files.exists(sourceDir)) {
                 System.out.println("Source data directory does not exist. Run
export first.");
                 return;
            }

            Files.createDirectories(backupDestDir);
```

```
            // Using Files.walk to copy files
            try (Stream<Path> stream = Files.walk(sourceDir)) {
                stream.forEach(source -> {
                    try {
                        Files.copy(source,
backupDestDir.resolve(sourceDir.relativize(source)),
StandardCopyOption.REPLACE_EXISTING);
                    } catch (IOException e) {
                        System.err.println("Failed to copy file: " + source +
" -> " + e.getMessage());
                    }
                });
            }
            System.out.println("Backup created successfully at: " +
backupDestDir);

            // Demonstrate recursive size calculation
            long size =
RecursiveFileUtils.calculateDirectorySize(backupDestDir);
            System.out.printf("Total size of backup directory '%s' is %d
bytes.\n", backupDestDir.getFileName(), size);

        } catch (IOException e) {
            System.err.println("Could not create backup: " + e.getMessage());
        }
    }
```

After:

```
public void createBackup() {

          AppConfig config = AppConfig.getInstance();
          Path sourceDir = Paths.get(config.getDataDirectory());
          String timestamp =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd_HH-mm-
ss"));
          Path backupParentDir = Paths.get(config.getBackupDirectory());
          Path backupDestDir = backupParentDir.resolve("backup_" + timestamp);

          try {
              if (!sourceDirectoryExists(sourceDir)) {
                  return;
              }

              Files.createDirectories(backupDestDir);
              copyFilesRecursively(sourceDir, backupDestDir);
              printBackupSuccess(backupDestDir);

          } catch (IOException e) {
              System.err.println("Could not create backup: " +
e.getMessage());
          }
      }

      private boolean sourceDirectoryExists(Path sourceDir) {
          if (!Files.exists(sourceDir)) {
```

```
                System.out.println("Source data directory does not exist. Run
export first.");
                return false;
            }
            return true;
        }

        private void copyFilesRecursively(Path sourceDir, Path backupDestDir)
throws IOException {
            try (Stream<Path> stream = Files.walk(sourceDir)) {
                stream.forEach(source -> copyFile(source, sourceDir,
backupDestDir));
            }
        }

        private void copyFile(Path source, Path sourceDir, Path backupDestDir) {
            try {
                Files.copy(source,
backupDestDir.resolve(sourceDir.relativize(source)),
                        StandardCopyOption.REPLACE_EXISTING);
            } catch (IOException e) {
                System.err.println("Failed to copy file: " + source + " -> " +
e.getMessage());
            }
        }

        private void printBackupSuccess(Path backupDestDir) {
            System.out.println("Backup created successfully at: " +
backupDestDir);
            long size =
RecursiveFileUtils.calculateDirectorySize(backupDestDir);
            System.out.printf("Total size of backup directory '%s' is %d
bytes.\n",
                    backupDestDir.getFileName(), size);
        }
```

2.

File: src/edu/ccrm/service/ReportService.java

Before:

```
private double calculateGpa(Student student) {
        double totalPoints = 0;
        int totalCredits = 0;
        for (Enrollment enrollment : student.getEnrolledCourses()) {
            if (enrollment.getGrade().getGradePoint() >= 0) { // Exclude 'NA'
grades
                totalPoints += enrollment.getGrade().getGradePoint()
                             * enrollment.getCourse().getCredits();
                totalCredits += enrollment.getCourse().getCredits();
```

```
            }
        }
        return totalCredits == 0 ? 0.0 : totalPoints / totalCredits;
    }
```

After:

```
private static final double valid_grade_threshold = 0.0;
    private double calculateGpa(Student student) {
        double totalPoints = 0;
        int totalCredits = 0;

        for (Enrollment enrollment : student.getEnrolledCourses()) {
            if (hasValidGrade(enrollment)) {
                totalPoints += enrollment.getGrade().getGradePoint() *
enrollment.getCourse().getCredits();
                totalCredits += enrollment.getCourse().getCredits();
            }
        }
        return totalCredits == 0 ? 0.0 : totalPoints / totalCredits;
    }
    private boolean hasValidGrade(Enrollment enrollment) {
       return enrollment.getGrade().getGradePoint() >= valid_grade_threshold;
    }
```

3.

File: src/edu/ccrm/cli/helpers/CourseManagementHelper.java

Before:

```
private void filterCourses(Scanner scanner) {
        System.out.println("Filter by: 1. Instructor, 2. Department, 3. Semester");
        int filterChoice = InputValidator.getInt(scanner, "Enter filter type: ");
        Predicate<Course> filter = course -> true;     // Default: no filter

        if (filterChoice == 1) {
            String name = InputValidator.getString(scanner, "Enter Instructor Name: ");
            filter = CourseService.filterByInstructor(name);
        } else if (filterChoice == 2) {
            String dept = InputValidator.getString(scanner, "Enter Department: ");
            filter = CourseService.filterByDepartment(dept);
        } else if (filterChoice == 3) {
            String semStr = InputValidator.getString(scanner, "Enter Semester (SPRING, FALL,
etc.): ");
            try {
                Semester sem = Semester.valueOf(semStr.toUpperCase());
                filter = CourseService.filterBySemester(sem);
            } catch (IllegalArgumentException e) {
                System.err.println("Invalid semester.");
                return;
            }
        }

        System.out.println("\n--- Filtered Courses ---");
        courseService.searchCourses(filter).forEach(System.out::println);
    }
```

After:

```
private void filterCourses(Scanner scanner) {
        System.out.println("Filter by: 1. Instructor, 2. Department, 3. Semester");
        int filterChoice = InputValidator.getInt(scanner, "Enter filter type: ");
        Predicate<Course> filter = getNoFilterPredicate();

        if (filterChoice == 1) {
            String name = InputValidator.getString(scanner, "Enter Instructor Name: ");
            filter = CourseService.filterByInstructor(name);
        } else if (filterChoice == 2) {
            String dept = InputValidator.getString(scanner, "Enter Department: ");
            filter = CourseService.filterByDepartment(dept);
        } else if (filterChoice == 3) {
            String semStr = InputValidator.getString(scanner, "Enter Semester (SPRING, FALL,
etc.): ");
            try {
                Semester sem = Semester.valueOf(semStr.toUpperCase());
                filter = CourseService.filterBySemester(sem);
            } catch (IllegalArgumentException e) {
                System.err.println("Invalid semester.");
                return;
            }
        }

        System.out.println("\n--- Filtered Courses ---");
        courseService.searchCourses(filter).forEach(System.out::println);
    }
    private Predicate<Course> getNoFilterPredicate() {
        return course -> true;
    }
```